CS2106 Introduction to Operating Systems
Lab 3 - Synchronization Mechanisms
Week of 21 September 2026 (Initial Release)
Week of 12 October, 2026 (Demos – Segment A)
Week of 19 October, 2026 (Demos – Segment B)

Instructions

a. For lab 3, the order of demonstration is A first again: Students in segment A will demo in **week 9**, and students in segment B will demo in week 10. Note again that there is the slight overlap in schedule, so demos start in week 9.

Lab #	Release in week	Demo Week One | Demo Segment	Demo Week Two | Demo Segment
#03	8	9 | A	10 | B

b. The lab weightage is **7 marks.**

Introduction

In this lab we will look at synchronization mechanisms to help parallel programs operate correctly. In the first part we will look at the idea of lock variables, and of how to use semaphores. In the second part we will implement barriers using semaphores, and in the third part we will use barriers to help find the smallest and largest elements of a moderately sized array.

As an added challenge we will be using the synchronization mechanisms with processes rather than threads, and we will see how to correctly share mechanisms like semaphores between processes.

Note before starting the lab:

If you are running this on the compute cluster, when obtaining a job allocation, you will want to run the following salloc commands.

For part 1:
```
salloc -c 5
```
For part 2:
```
salloc -c 7
```
For part 3:
```
salloc -C xcnf -c 8
```

Essentially, the -c requests the allocation for more CPUs to be allocated (so you can see the parallelism), while -C requests the allocation to be on the xcnf machines instead. Otherwise, it usually defaults CPU jobs to xcnd, which is much older hardware and will take a little longer to run the prime counting code.

Part 1. Using Lock Variables and Semaphores

All files needed for this part can be found in the part1 directory. In this part we will look at two tools for synchronization between processes that we have seen in lectures and tutorials: lock variables and semaphores. First, take a look at lab3p1.c.

```c
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <sys/shm.h>
#include <sys/wait.h>

#define NUM_PROCESSES 5

int main() {

    int i, j, pid;

    for(i=0; i<NUM_PROCESSES; i++)
    {
        if((pid = fork()) == 0) {
            break;
        }
    }

    if(pid == 0) {
        printf("I am child %d\n", i);

        for(j = i*10; j<i*10 + 10; j++){
            printf("%d ", j);
            fflush(stdout);
            usleep(250000);
        }

        printf("\n\n");
    }
    else {
        for(i=0; i<NUM_PROCESSES; i++)
            wait(NULL);
    }

}
```

This is a simple program that spawns 5 processes, where the goal is to produce the following output:

```
I am child 0
0 1 2 3 4 5 6 7 8 9

I am child 1
10 11 12 13 14 15 16 17 18 19

I am child 2
20 21 22 23 24 25 26 27 28 29

I am child 3
30 31 32 33 34 35 36 37 38 39

I am child 4
40 41 42 43 44 45 46 47 48 49
```

Compile and run the program.

```
gcc lab3p1.c –o lab3p1
./lab3p1
```

Note: If on the compute cluster, slurm also does its own buffering of output to terminal. So you may want to use

```
srun –u ./lab3p1
```

otherwise, you will only see the outputs appear in chunks.

As you should expect by now, instead of getting the intended output, we get a jumbled combination of prints by the separate processes. This is an example run obtained:

```
I am child 0
0 I am child 1
10 I am child 2
20 I am child 3
30 I am child 4
40 1 11 21 31 41 2 12 22 32 42 3 13 23 33 43 14 4 24 34 44 15 5 25 35 45 16 6 26 36 46 17 7 27 37 47 18 8 28 38 48 19
9 29 39 49
```

Observe that the “<some number> I am child X” (X is some number from 0 to 4) appears before the rest of the numbers, as well as the observation that the numbers printed by each child are interleaved.

a. Using Lock Variables

We will now explore a mechanism called “lock variables” to try to synchronize the different processes. Lock variables are very simple and following pseudocode is how we might try to use them.

```
Create a new shared variable “lock”. Set it to 1
For each process:
    while (lock is 0) Do nothing.  (Busy wait until lock is 1)
    Set lock to 0        //we can only get here when lock is 1, thus passing the while loop
    Execute my code  // we have the lock
```

This is implemented in lab3p1-lock.c.

Note that the lock variable does not control which process gets to run first. The goal at this point is to get it to do something like the following:

```
I am child 0
0 1 2 3 4 5 6 7 8 9

I am child 3
31 32 33 34 35 36 37 38 39
etc.
```

That is, we want one process to complete running before the next one starts.

Compile and run lab3p1-lock.c by using:

```
gcc lab3p1-lock.c –o lab3p1lock
./lab3p1lock
```

If you try running this (may require a couple of tries) and look at the result of our program, it is clear we have not achieved our objective of ensuring only one process runs at a time. Think about why the lock variable failed to coordinate the processes.

b. Using Turn Variables

You've already seen the concept of "turn variables" in Tutorial 4 Question 4. To summarize:

```
Create a shared variable called "turn". Set turn to 0.
For each process i from 0 to n-1:
    while(turn != i);  // Do nothing while it is not our turn.
    //Now it is my turn
    //Execute my code here.
    turn = turn + 1; // Pass turn to next process
```

You are given a file called lab3p1-shm.c. Modify this program to implement turn variables. Does it work correctly? What might be a disadvantage of turn variables?

c. Using Semaphores

We will now attempt to use semaphores. Open up sema-wrong.c (yes, the solution is indeed wrong) and you will see a straightforward but naive attempt to use semaphores to coordinate between processes:

```c
sem_t sem;

int main() {
    int pid;

    sem_init(&sem, 1, 0);

    if((pid = fork()) != 0) {
        printf("Parent! Making my child wait for 1 second.\n");
        sleep(1);
        sem_post(&sem);
    }
    else
    {
        sem_wait(&sem);
        printf("Child! Waited 1 second for parent.\n");
    }

    if(pid != 0)
    {
        wait(NULL);
        sem_destroy(&sem);
    }
}
```

Compile and run this program.

```
gcc sema-wrong.c -o sema-wrong -lpthread
./sema-wrong
```

You will observe that the program gets stuck. There is no shared memory between the parent and child process. Therefore, the semaphore being posted by the parent is different from the semaphore waited on by the child. Hence the child remains waiting.

We will now see how to correctly share semaphores between processes. Open up sema-right.c and you will see:

```c
int main() {
    int shmid, pid;
    sem_t *sem;

    shmid = shmget(IPC_PRIVATE, sizeof(sem_t), IPC_CREAT | 0600);
    sem = (sem_t *) shmat(shmid, NULL, 0);

    sem_init(sem, 1, 0);

    if((pid = fork()) != 0) {
        printf("Parent!. Making my child wait for 1 second.\n");
        sleep(1);
        sem_post(sem);
        wait(NULL);
        sem_destroy(sem);
        shmctl(shmid, IPC_RMID, 0);
    }
    else
    {
        sem_wait(sem);
        printf("Child! Waited 1 second for parent.\n");
    }
}
```

Notice what we've done here:

1. Our semaphore sem is of type sem_t * instead of sem_t.
2. We create a segment of shared memory using shmget of size sem_t.
3. We attach the shared memory to sem using shmat.
4. From then on we treat sem as an ordinary semaphore. Note that now we pass in sem instead of &sem to sem_wait and sem_post, since sem is now of type sem_t * instead of sem_t.

5. We destroy the semaphore, then detach and release the shared memory once it is no longer needed.

Compile and run this program.

```
gcc sema-right.c –o sema-right –lpthread
./sema-right
```

We now see that the program runs correctly; the parent pauses for a second before releasing the child and we see that it works correctly:

```
Parent!. Making my child wait for 1 second.
Child! Waited 1 second for parent.
```

You are now given lab3p1-sem.c, which currently is identical to lab3p1.c. Using what you've learnt about semaphores, modify lab3p1-sem.c to produce this output, with the correct ordering of processes from 0 to 4:

```
I am child 0
0 1 2 3 4 5 6 7 8 9

I am child 1
10 11 12 13 14 15 16 17 18 19

I am child 2
20 21 22 23 24 25 26 27 28 29

I am child 3
30 31 32 33 34 35 36 37 38 39

I am child 4
40 41 42 43 44 45 46 47 48 49
```

DEMO 1 (3 marks)

Compile your lab3p1-sem.c and demonstrate to your TA that it works correctly, and show them that you indeed used semaphore(s) to achieve this.

Part 2. Creating Barriers with Semaphores

In this part we will create a barrier using semaphores. Recall that a barrier is a structure that is called by an expected number of processes. All processes except the last will block at the barrier, until the last process calls the barrier, after which all processes are released. All files needed can be found in the part2 directory.

When we create barriers using semaphores, we are relying on the following property of semaphores:

Initialize sem = 0
If *n* processes call wait(sem), all *n* processes will block.
If another process calls signal(sem), ONE of the *n* processes will be picked to unblock.
If each of the *n* processes also calls signal(sem) after being unblocked, eventually everyone will be unblocked.

Using the principle above, we can design a barrier using the following pseudo-code (note: This is incomplete and just gives you an idea of how a barrier works):

```
int nproc = 0, count = 0;
sem_t barrier;

// Initializes the barrier
void init_barrier(int num_proc) {
      nproc = num_proc;
      count = 0;
      Initialize barrier to 0.
}

// Every process calls this to "reach" the barrier

void reach_barrier() {
      count++;
      if(count == nproc) {
            // Release the semaphore
            signal(barrier);
      }
      else {
            // We are not the last process. So we wait at the
            // semaphore until we are freed.
            wait(barrier);

            // Now that we are freed, we free the next process
            signal(barrier);
      }
}
```

You are given three files barrier.c, barrier.h and test_barrier.c. The barrier.c file contains three functions:

`init_barrier`: Takes one argument – The number of processes that will call the barrier. This function should create any shared memory required, as well as the semaphores you need. Note that in the algorithm above count is a shared variable that is updated by multiple processes and can thus create race conditions. You will need one more semaphore to act as a mutex to protect this variable.

`reach_barrier`: Does not take any arguments. Follows the algorithm above.

`destroy_barrier`: Destroy all semaphores and detach and release all shared memory.

The barrier.h file contains prototypes for barrier.c, and test_barrier.c contains a test program that basically creates 6 children that will sleep a random amount of time of up to 1 second, then call reach_barrier. When all children reach the barrier, the parent will print "All the children have returned." and exit.

The children too will output "Child X has slept for Y seconds and has now reached the barrier".

After completing your barrier.c, you can compile and run using:

```
gcc barrier.c test_barrier.c –o test_barrier –lpthread
./test_barrier
```

If your barrier works correctly, you should see an output similar to this (sleep times will differ), the key is the parent prints that all children have returned last:

```
**Parent waiting for children**

        Child 4 slept for 0.12 seconds and has now reached the barrier
        Child 3 slept for 0.12 seconds and has now reached the barrier
        Child 1 slept for 0.40 seconds and has now reached the barrier
        Child 2 slept for 0.45 seconds and has now reached the barrier
        Child 5 slept for 0.59 seconds and has now reached the barrier
        Child 0 slept for 0.93 seconds and has now reached the barrier

**All the children have reached the barrier**
```

Test and make sure your barrier implementation works, including that your destroy barrier correctly detaches and frees shared memory (especially on the compute cluster!).

Part 3. Making use of barriers

The barrier is a surprisingly useful synchronization mechanism with many applications. One very common application is when parallelizing computation. Suppose we have a large array of numbers, and we want to count how many primes there are. This can be done sequentially like follows:

```
initialize count = 0
for each number in array do
    if (isPrime(number))
        count++
```

Since checking if a number in the array is prime is independent of the other checks, it is trivial to parallelize this program by doing the following. Suppose we want to run *k* processes. Then we break the array up into *k* chunks, and each processes computes the number of primes in its chunk.

In this part we will go through a moderately sized 2,000,000 element array to find the number of primes among its elements. You are given two programs and test case generator. Most of what you need can be

found in the part3 directory, although you will need to copy over the barrier.* files from the previous part to complete your program.

a. A code to generate two test cases. It uses a fixed seed so the output will be the same each time it is run. (This is just to keep the distribution files small.) Compile it and run (might take a few seconds), and you will see test1.in and test2.in be created.

```
gcc generator.c -o generator
./generator
```

b. The sequential version primecount.c. Compile and run this program:

```
gcc primecount.c -o primecount
./primecount < test1.in
```

After a few seconds, you should see an output like follows (time taken will differ based on the hardware):

```
Number of items: 2000000
Number of primes is 195448
Time taken is 4.186 seconds
```

You can use this to compare with the parallel version's output. This helps you to check if your program is working correctly.

c. The (incorrectly implemented) parallel version, primecount-par.c. This program splits the 2,000,000-element array between 8 processes (250,000 integers per process). There is also an array "counts" where each process will store its results (e.g. process 0 will store to counts[0] etc.). The parent process should sum through count after all processes are done to find the total number of primes.

Compile and execute using:

```
gcc primecount-par.c -o primecount-par
./primecount-par
```

You will see an output like following (actual number of primes may differ slightly, and can even differ from run to run!):

```
Number of items: 2000000
Number of primes is 91605
Time taken is 0.219 seconds
```

Comparing the number of primes found with the sequential implementation, this code is clearly incorrect.

Now modify primecount-par.c to produce the correct answer.

a. The program will still distribute the search between 8 processors.
b. You must use barriers to coordinate.
c. You must find the same number of primes.
d. Leave the code that measures the time taken to do the search in its current place in the code.

Answer the following questions:

DEMO 2 (4 marks)

Demo to your TA that your code does produce the correct answers for the given sample cases.

Note that while in this case, the parent needs to wait for the child processes to be done, in practice another common scenario is that all *k* processes perform some computation, and all must wait until all *k* processes are done before progressing to future computation steps so that all the results are available.